

API Documentation

1. Authentication & Authorization APIs

Base URL: **http://localhost:5000/api/**

Description: Handles secure access using JWT and Bcrypt hashing.

POST /register

Description: Creates a new user account.

Authentication: Public.

Request Headers: Content-Type: application/json

Request Body Parameters:

- username (String): Unique identifier.
- email (String): Valid email address.
- password (String): Plain text password.
- role (String): **One of** (community_member, facility_manager, worker, admin.)

Example Request:

```
{ "username": "admin1", "email": "admin1@example.com", "password": "123456",  
  "role": "admin" }
```

Example Success Response (201 Created):

```
{ "message": "User registered successfully", "user": { "user_id": 6, "username": "admin1",  
  "email": "admin1@example.com", "role": "admin", "is_active": true, "created_at": "2026-05-  
10T12:19:47.873Z" } }
```

POST /login

Description: Authenticates user and returns a JWT token.

Authentication: Public.

Request Body Parameters:

- username (String)
- password (String)

Example Request:

```
{ "email": "admin1@example.com", "password": "123456" }
```

Example Success Response (200 OK):

```
{ "message": "Login successful", "token":  
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NiwiZXN0IjE6ImFkbWluIiwiaWF0IjoxNzc4NDI2Mzk1LCJleHAiOiJlE3Nzg0MzM1OTV9.mbn2P1tAO8-  
vCEz7tWLEyxvGtDmQ1iS7XPC3YjpW80", "user": { "id": 6, "username": "admin1", "email":  
"admin1@example.com", "role": "admin", "is_active": true }}
```

POST /logout

Description: Invalidate the user's session/token on the client side.

Authentication: Token required.

Example Success Response (200 OK):

```
{"message": "Logged out successfully"}
```

POST /forgot-password

Description: Verifies the username exists in the database to initiate recovery.

Authentication: Public.

Request Body Parameters:

```
{ "email": "community1@example.com" }
```

Example Success Response (200 OK):

```
{"message": "Password reset email generated for community1@example.com",  
"preview": "https://ethereal.email/message/agCkF77r6kD8o-  
ruagCkGoIKErNPYURYAAAAAZk0mQw1qcbBmKViHnTYGgI"}
```

POST /reset-password

Description: Updates the password for a specific user after verification.

Authentication: Public.

Request Body Parameters

```
{ "email": "community1@example.com", "newPassword": "654321" }
```

Example Success Response (200 OK):

```
{ "message": "Password reset successfully", "user": { "user_id": 3, "username":  
"community1", "email": "community1@example.com", "role": "community_member" }}
```

Role-Based Access Control (RBAC)

The backend uses a two-gate middleware system to protect routes. This is implemented in `authMiddleware.js` and applied to routes in `issueRoutes.js` and `userRoutes.js`.

Gate 1: `verifyToken`

Checks the Authorization header for a Bearer <token>.

If missing or invalid, returns 401 Unauthorized.

Gate 2: authorizeRoles (...allowedRoles)

Extracts the role from the decoded JWT.

If the user's role is not in the allowedRoles list, returns 403 Forbidden.

2. Issue Management APIs

POST <http://localhost:5000/api/issues>

Description: Submit a new issue.

Authorization: Bearer COMMUNITY_TOKEN

Request Body Parameters: { "title": "Broken chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.", "location_id": 1, "category_id": 3 }

Example Success Response (201 OK):

```
{ "message": "Issue created successfully", "issue": { "ticket_id": 1, "title": "Broken chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.", "photo_url": null, "completed_photo_url": null, "location_id": 1, "category_id": 3, "status": "pending", "priority": "medium" "created_by": 3, "assigned_to": null, "created_at": "2026-05-10T12:32:53.282Z", "updated_at": "2026-05-10T12:32:53.282Z" }}
```

GET <http://localhost:5000/api/issues>

Description: Get all issues

Authorization: Bearer MANAGER_TOKEN

Example Success Response (200 OK):

```
{ "count": 1, "issues": [ { "ticket_id": 1, "title": "Broken chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.", "photo_url": null, "completed_photo_url": null, "location_id": 1, "category_id": 3, "status": "pending", "priority": "medium", "created_by": 3, "assigned_to": null, "created_at": "2026-05-10T12:32:53.282Z", "updated_at": "2026-05-10T12:32:53.282Z" } ] }
```

GET <http://localhost:5000/api/issues/my>

Description: Get issues submitted by the logged-in Community Member

Authorization: Bearer COMMUNITY_TOKEN

Example Success Response (200 OK):

```
{ "count": 1, "issues": [ { "ticket_id": 1, "title": "Broken chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.", "photo_url": null, "completed_photo_url": null, "location_id": 1, "category_id": 3, "status": "pending", "priority": "medium",
```

```
"created_by": 3, "assigned_to": null, "created_at": "2026-05-10T12:32:53.282Z",  
"updated_at": "2026-05-10T12:32:53.282Z"  ]}]}
```

GET <http://localhost:5000/api/issues/assigned/my>

Description: get all assigned issues

Authorization: Bearer WORKER_TOKEN

Example Success Response (200 OK):

```
{ "count": 0, "issues": [] }
```

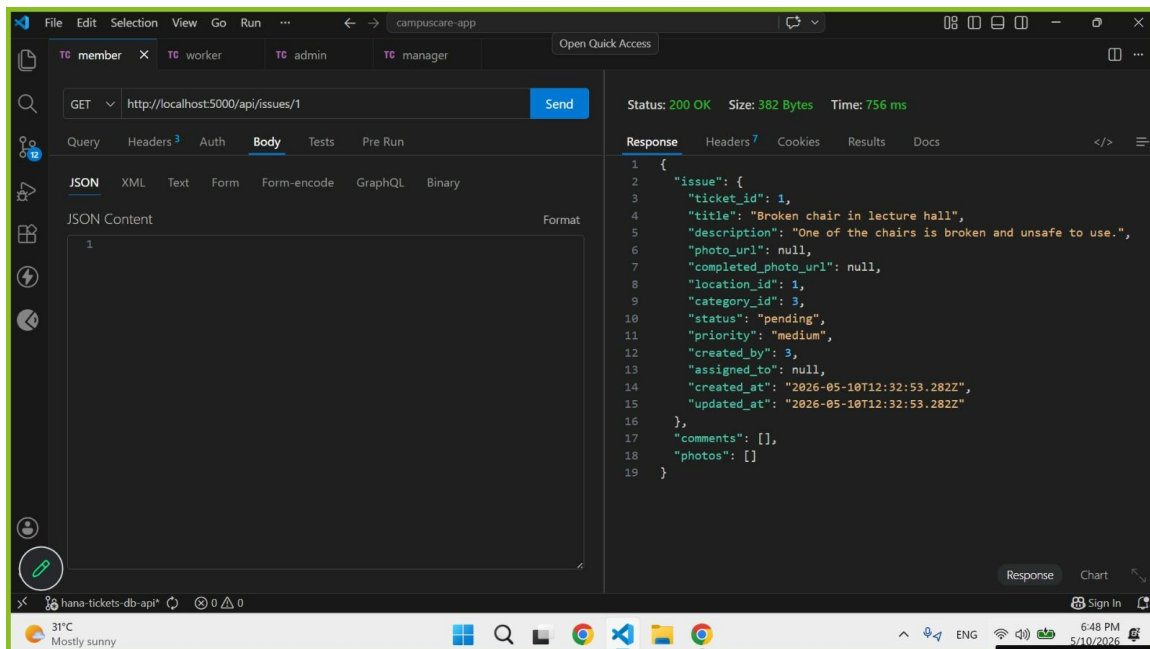
GET <http://localhost:5000/api/issues/1>

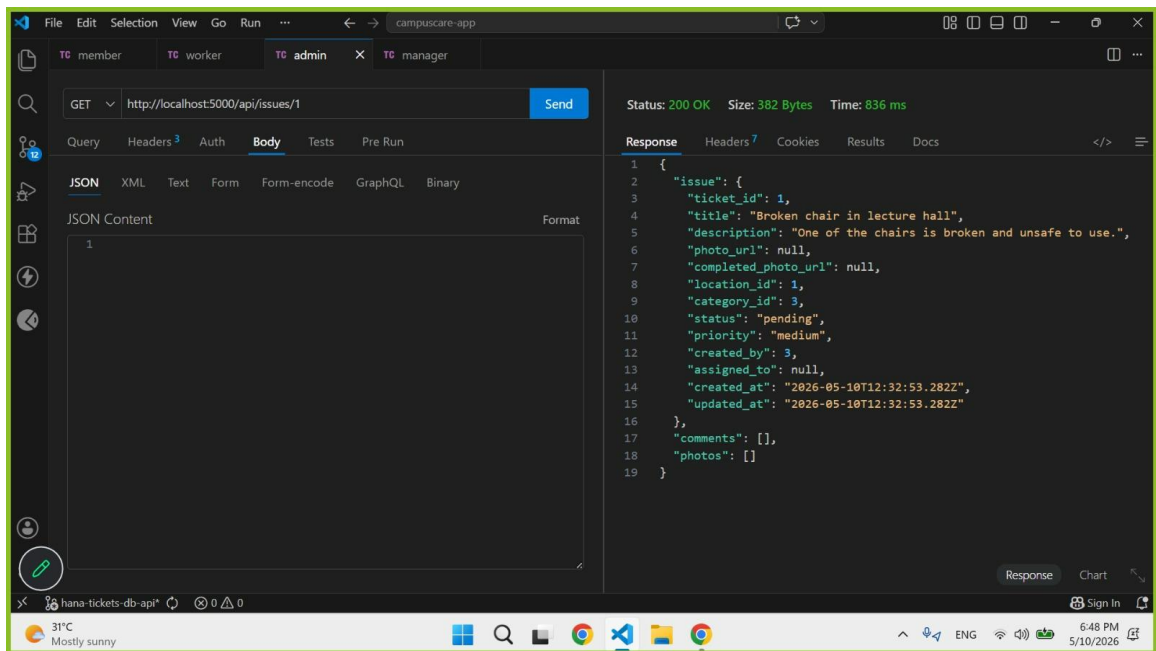
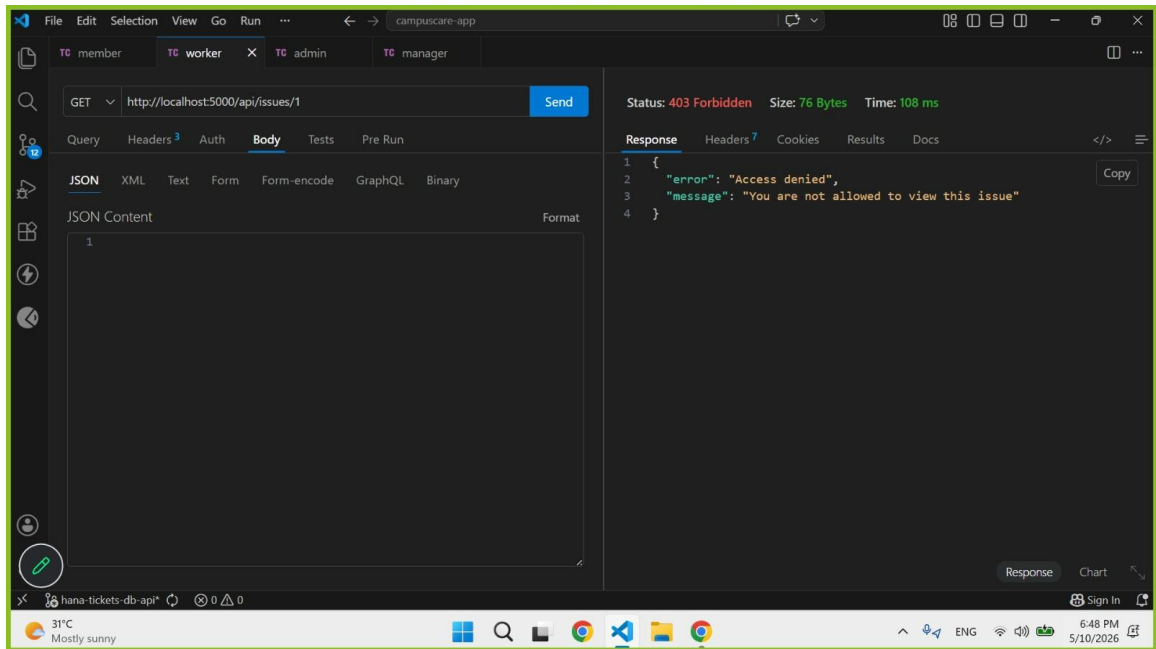
Description: Facility Manager: can view all issues.

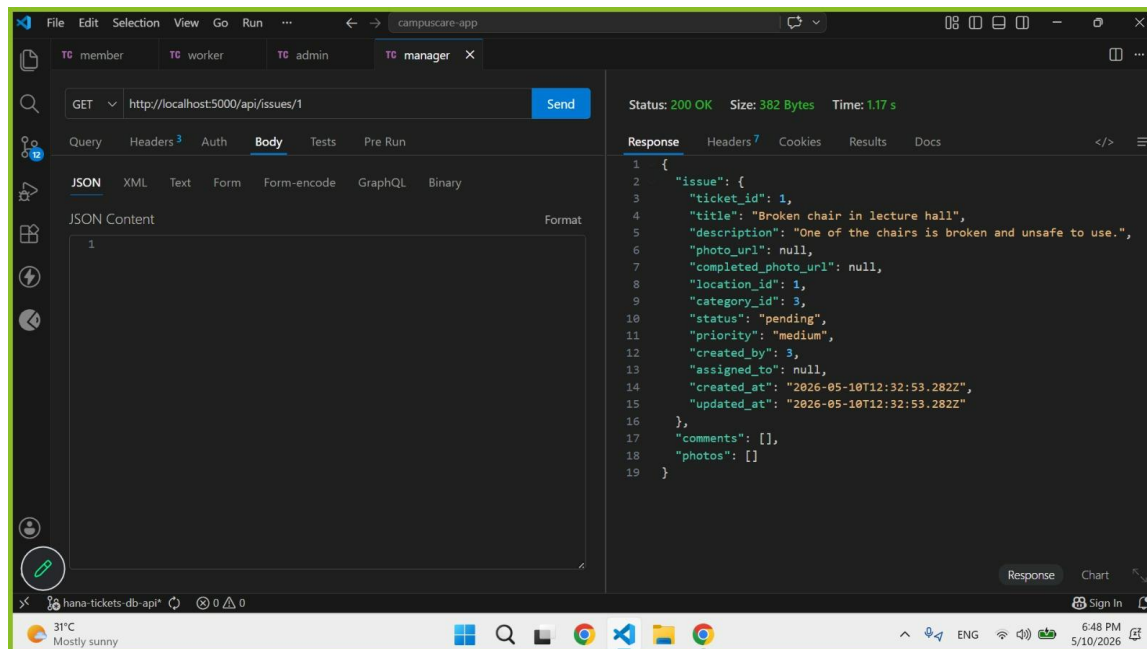
Admin: can view all issues.

Community Member: can view only their own submitted issues.

Worker: can view only issues assigned to them.







PUT <http://localhost:5000/api/issues/1/assign>

Description: Assign an issue to a worker (Facility Manager)

Request Body Parameters: { "assigned_to": 5 }

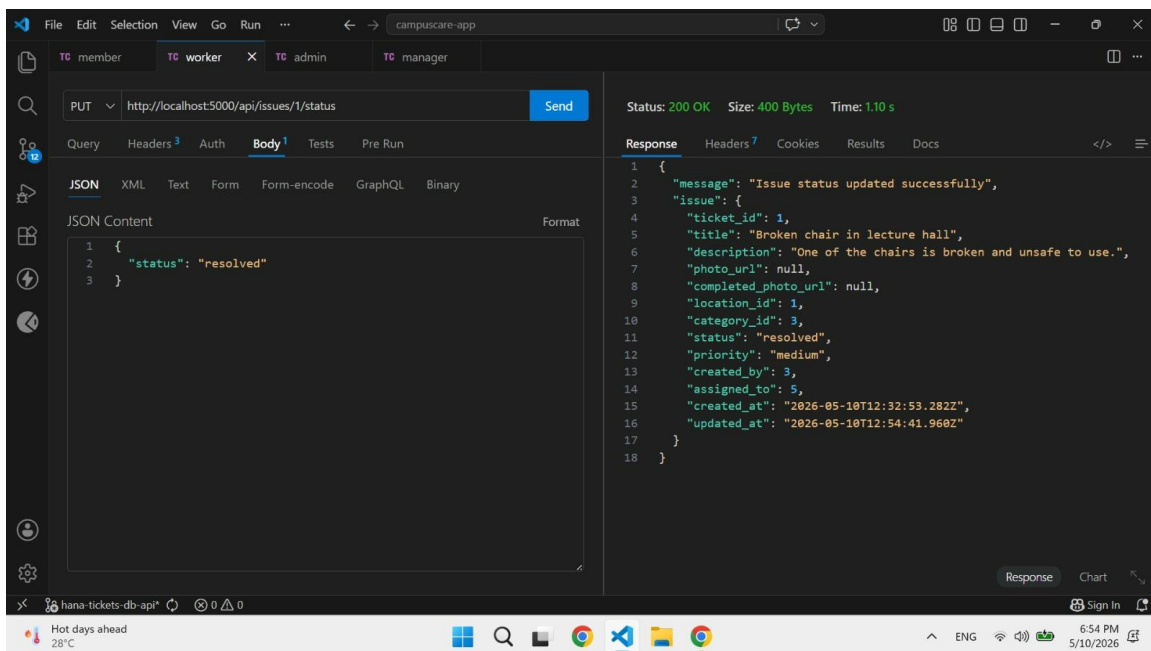
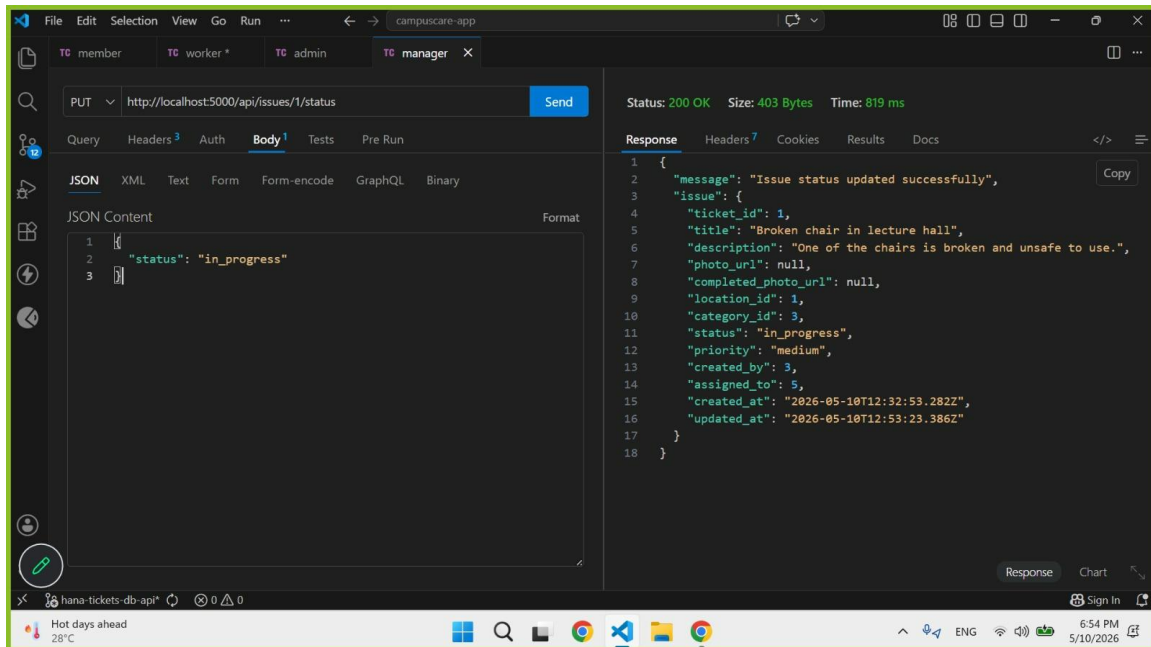
Authorization: Bearer MANAGER_TOKEN

Example Success Response (200 OK):

```
{"message": "Issue assigned successfully", "issue": { "ticket_id": 1, "title": "Broken chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.", "photo_url": null, "completed_photo_url": null, "location_id": 1, "category_id": 3, "status": "assigned", "priority": "medium", "created_by": 3, "assigned_to": 5, "created_at": "2026-05-10T12:32:53.282Z", "updated_at": "2026-05-10T12:50:57.000Z" }}
```

PUT <http://localhost:5000/api/issues/1/status>

Description: Facility Manager can update status ,Worker can only update assigned issues.,Worker can only set status to in_progress or resolved



PUT <http://localhost:5000/api/issues/1/priority>

Description: update issue priority

Request Body Parameters { "priority": "high" }

Authorization: Bearer MANAGER_TOKEN

Example Success Response (200 OK):

```
{ "message": "Issue priority updated successfully", "issue": { "ticket_id": 1, "title": "Broken chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.", "photo_url": null, "completed_photo_url": null, "location_id": 1, "category_id": 3, "status": "resolved", "priority": "high", "created_by": 3, "assigned_to": 5, "created_at": "2026-05-10T12:32:53.282Z", "updated_at": "2026-05-10T12:56:33.337Z" }}
```

PUT <http://localhost:5000/api/issues/1/close>

Description: Close a resolved issue (Facility Manager)

Authorization: Bearer MANAGER_TOKEN

Example Success Response (200 OK):

```
{ "message": "Issue closed successfully", "issue": { "ticket_id": 1, "title": "Broken chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.", "photo_url": null, "completed_photo_url": null, "location_id": 1, "category_id": 3, "status": "closed", "priority": "high", "created_by": 3, "assigned_to": 5, "created_at": "2026-05-10T12:32:53.282Z", "updated_at": "2026-05-10T12:58:01.843Z" } }
```

POST <http://localhost:5000/api/issues/1/comments>

Description: Add a comment to an issue (Worker)

Request Body Parameters: { "comment_text": "I checked the issue and started fixing it." }

Authorization: Bearer WORKER_TOKEN

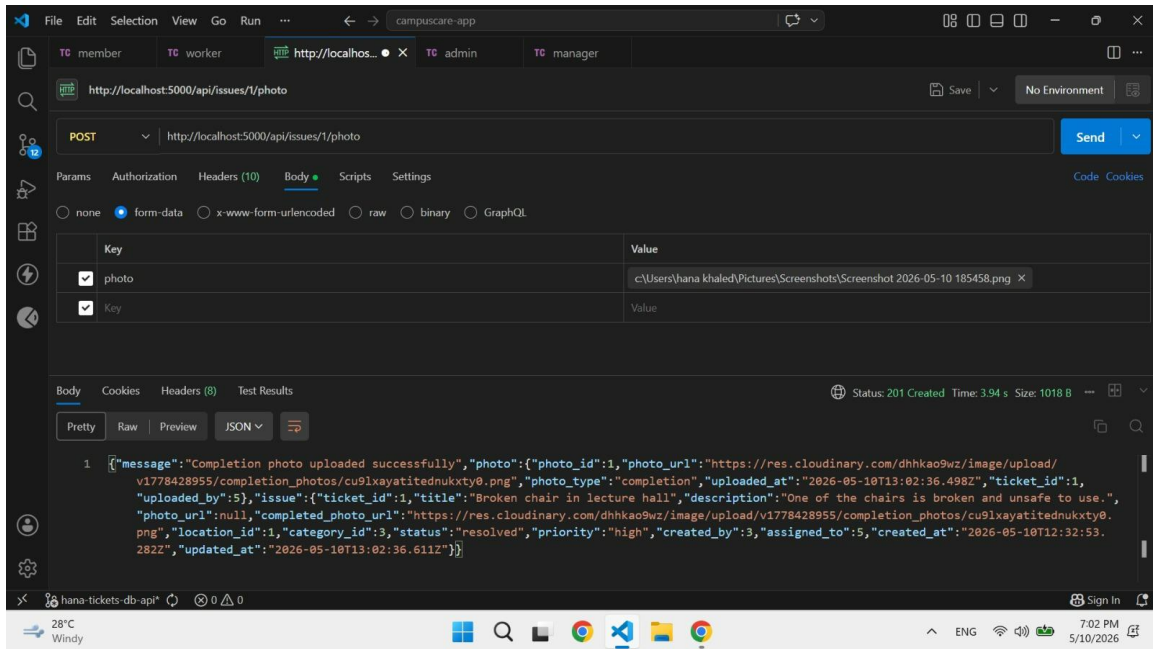
Example Success Response (201 OK):

```
{ "message": "Comment added successfully", "comment": { "comment_id": 1, "comment_text": "I checked the issue and started fixing it.", "created_at": "2026-05-10T13:00:20.092Z", "ticket_id": 1, "user_id": 5 } }
```

POST <http://localhost:5000/api/issues/1/photo>

Description: Upload a completion photo (Worker)

Authorization: Bearer WORKER_TOKEN



DELETE <http://localhost:5000/api/issues/1>

Description: Delete an issue (Facility Manager)

Authorization: Bearer MANAGER_TOKEN

Example Success Response (200 OK):

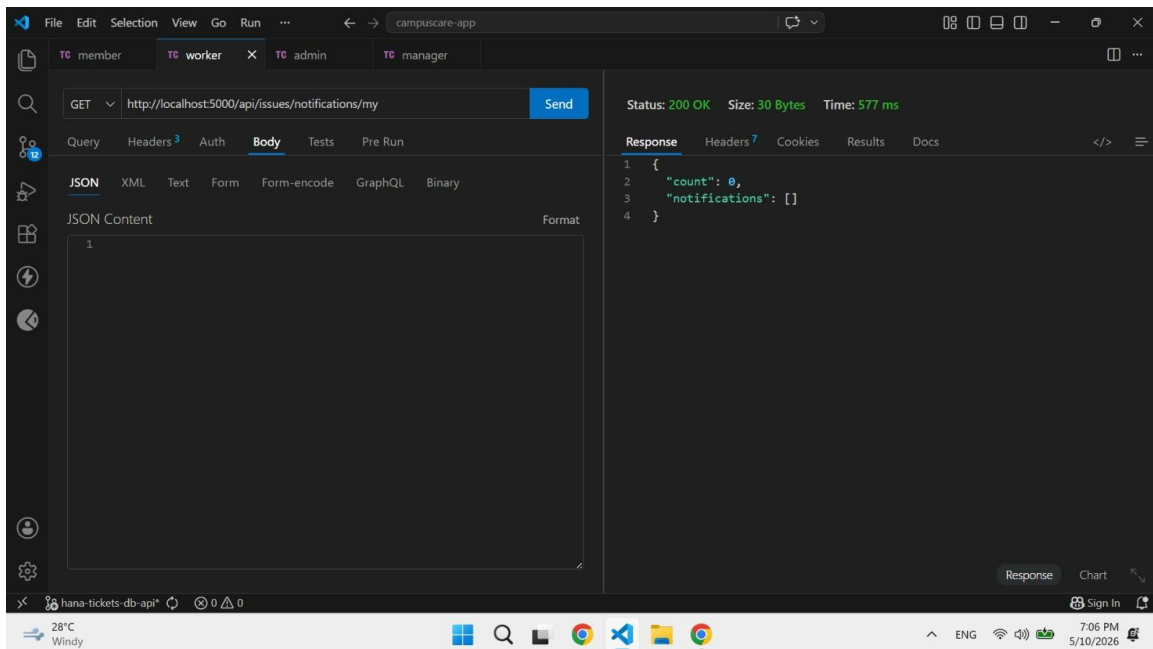
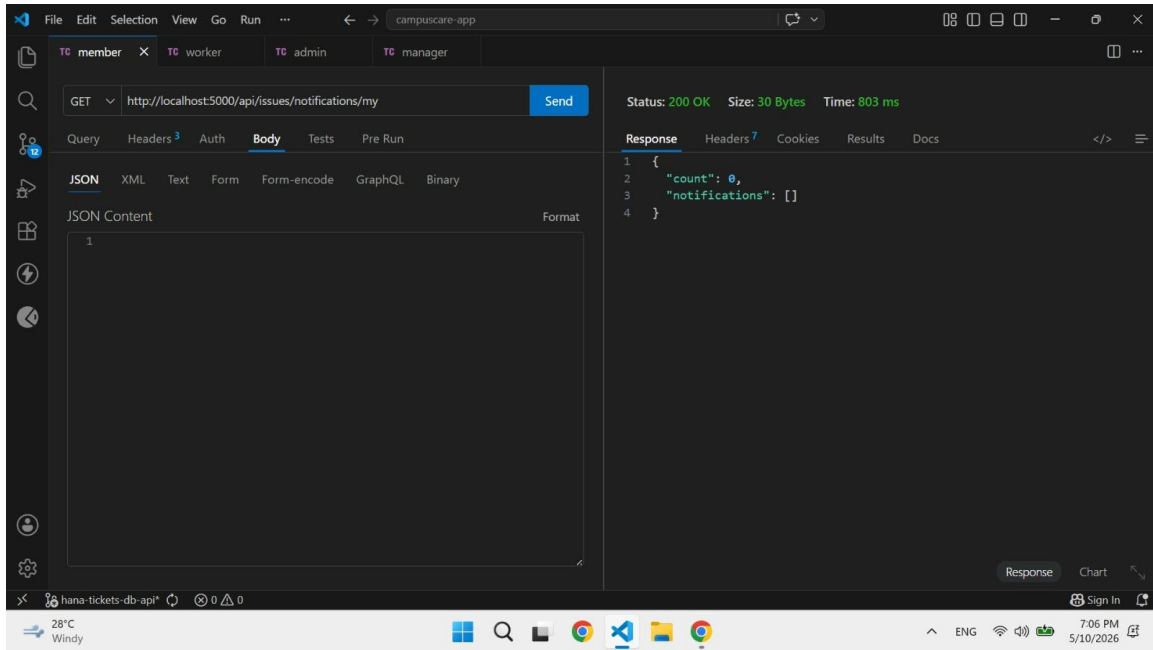
```

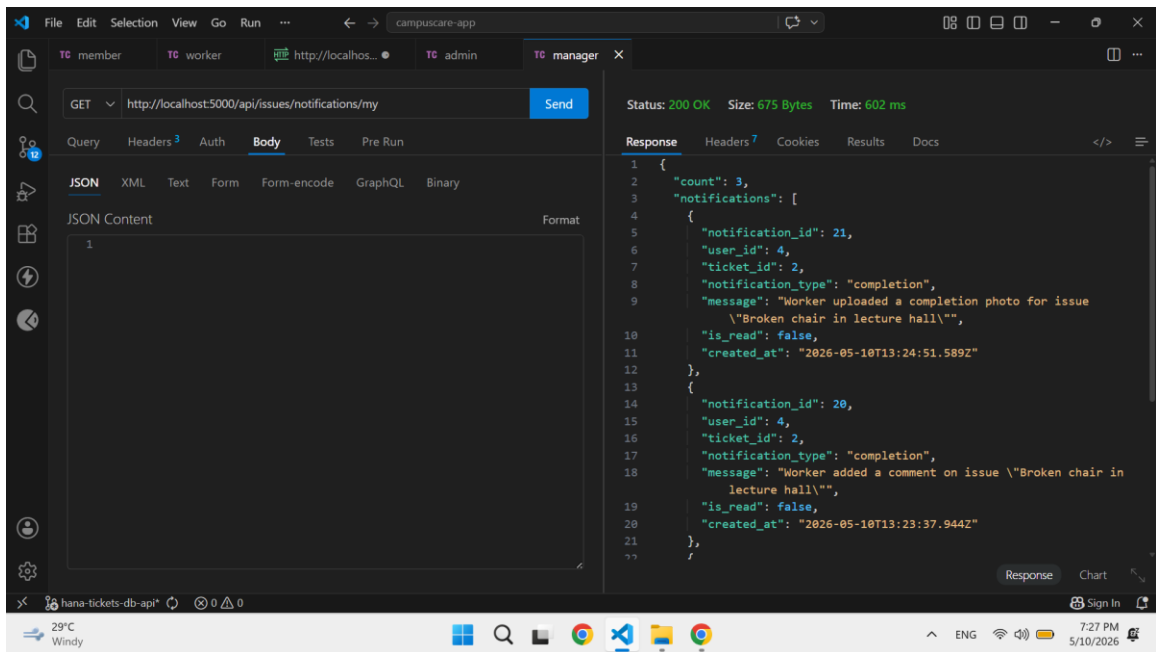
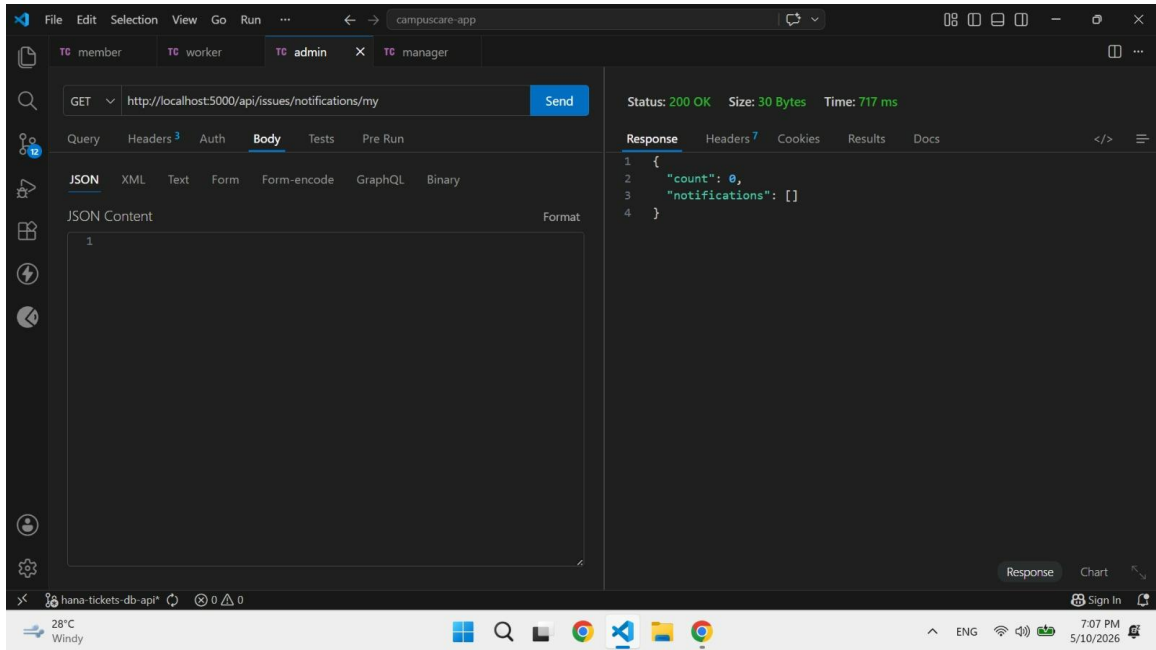
{"message": "Issue deleted successfully", "deletedIssue": { "ticket_id": 1, "title": "Broken
chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.",
"photo_url": null, "completed_photo_url":
"https://res.cloudinary.com/dhhkao9wz/image/upload/v1778428955/completion_photos
/cu9lxayatitednukxty0.png", "location_id": 1, "category_id": 3, "status": "resolved",
"priority": "high", "created_by": 3, "assigned_to": 5, "created_at": "2026-05-
10T12:32:53.282Z", "updated_at": "2026-05-10T13:02:36.611Z" }}

```

GET <http://localhost:5000/api/issues/notifications/my>

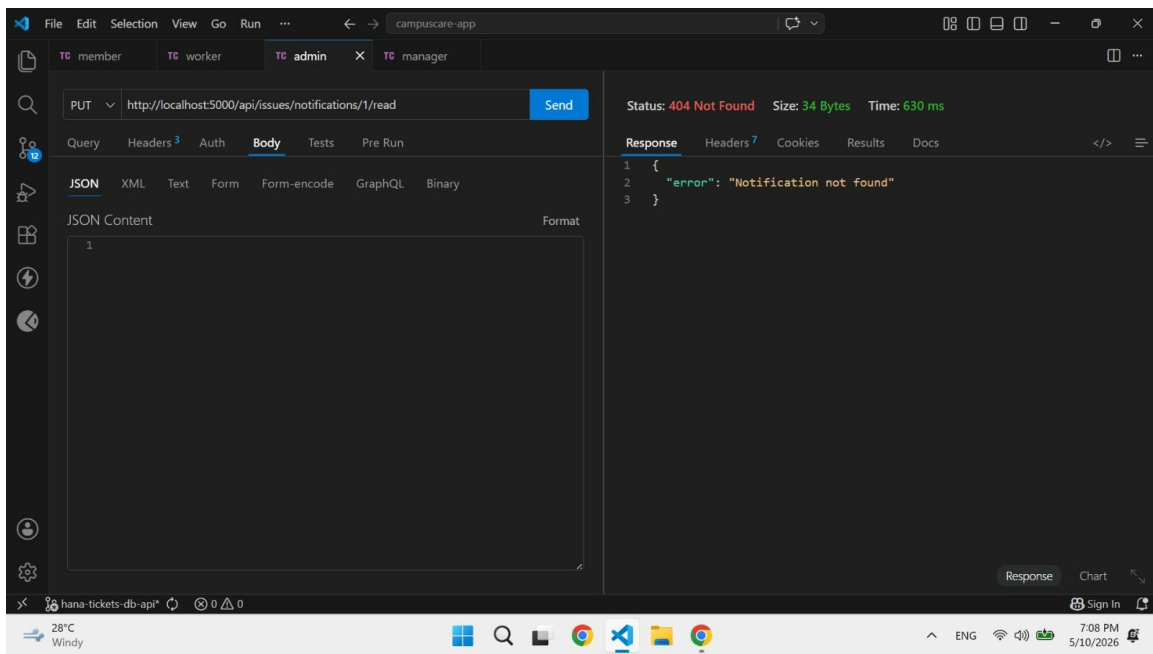
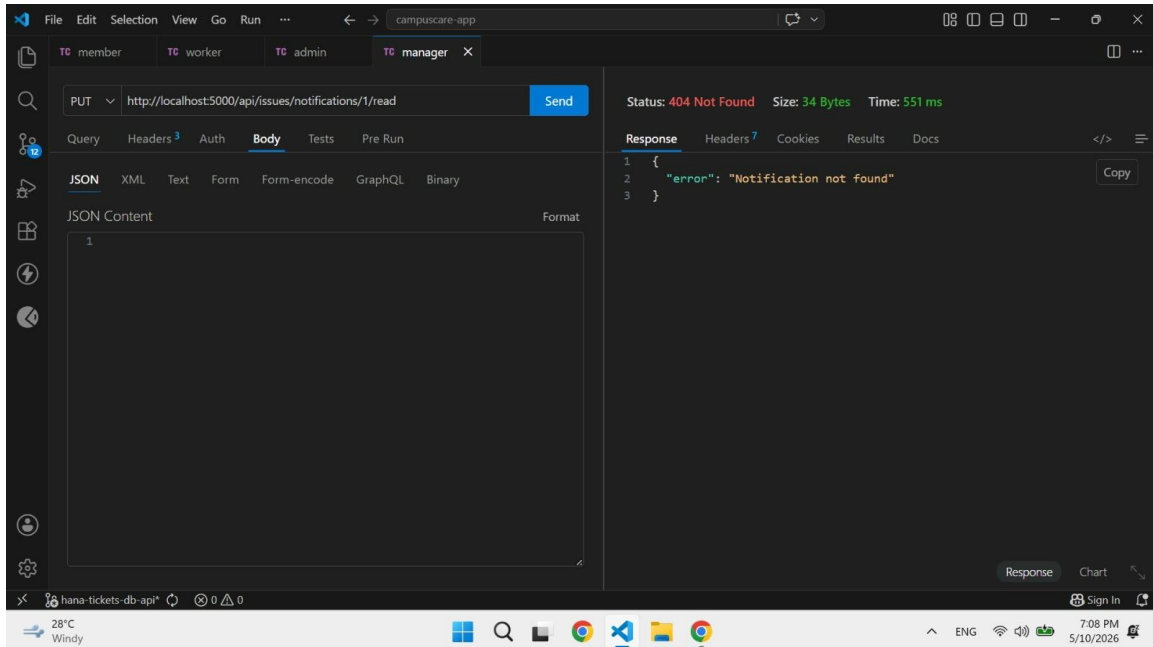
Description: get any login user notification

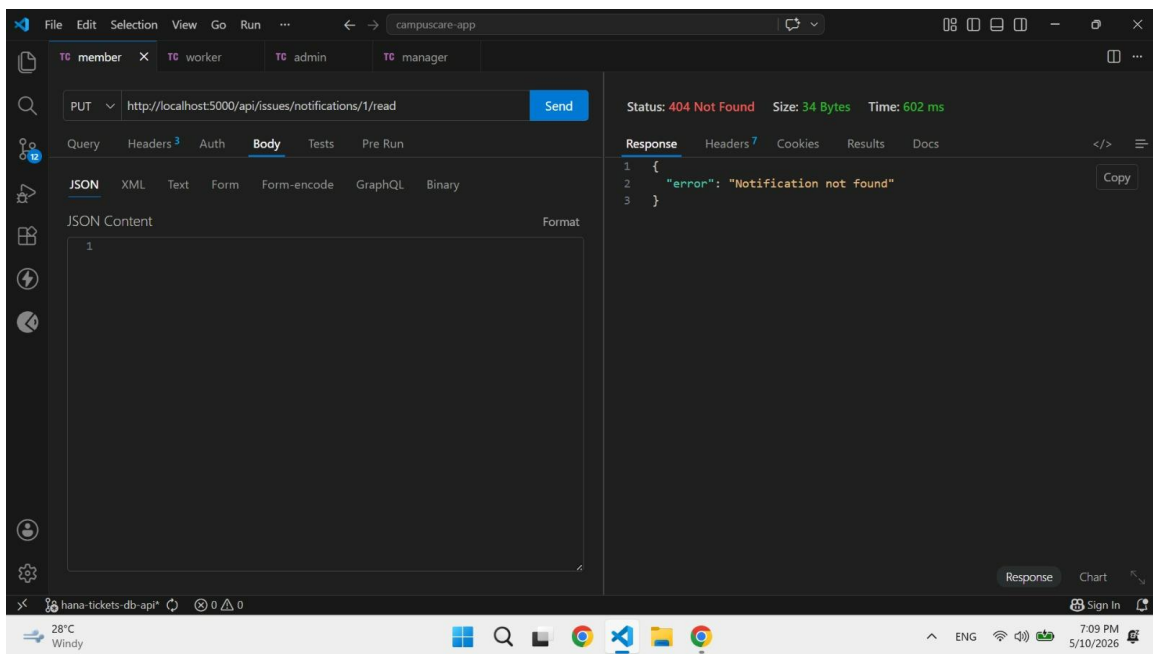
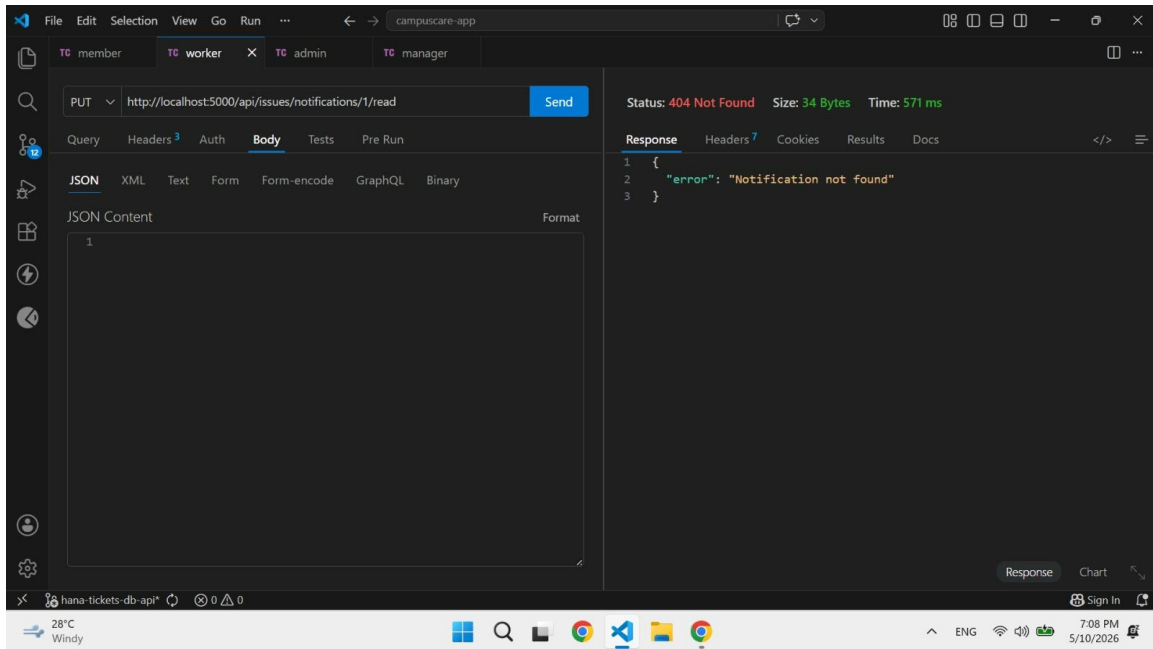




PUT <http://localhost:5000/api/issues/notifications/1/read>

Description: error since there is no notification right now , the user marks their own notification as read





GET <http://localhost:5000/api/manager/workers>

Description: List all worker accounts (Facility Manager)

Authorization: Bearer MANAGER_TOKEN

Example Success Response (200 OK):

```
{ "count": 1, "workers": [ { "user_id": 5, "username": "worker1", "email":  
"worker1@example.com", "role": "worker", "is_active": true, "created_at": "2026-05-  
10T12:19:14.900Z" } ] }
```

PUT <http://localhost:5000/api/manager/workers/5/status>

Description: Activate or deactivate a worker account

(Facility Manager)

Authorization: Bearer MANAGER_TOKEN

Request Body Parameters: { "is_active": false }

Example Success Response (200 OK):

```
{ "message": "Worker account deactivated successfully", "worker": { "user_id": 5,
"username": "worker1", "email": "worker1@example.com", "role": "worker", "is_active":
false}}
```

GET <http://localhost:5000/api/admin/users>

Description: List all registered users

Authorization: Bearer ADMIN_TOKEN

Example Success Response (200 OK):

```
{ "count": 5, "users": [ { "user_id": 6, "username": "admin1", "email":
"admin1@example.com", "role": "admin", "is_active": true, "created_at": "2026-05-
10T12:19:47.873Z" }, { "user_id": 5, "username": "worker1", "email":
"worker1@example.com", "role": "worker", "is_active": false, "created_at": "2026-05-
10T12:19:14.900Z" }, { "user_id": 4, "username": "manager1", "email":
"manager1@example.com", "role": "facility_manager", "is_active": true, "created_at": "2026-
05-10T12:18:35.112Z" }, { "user_id": 3, "username": "community1", "email":
"community1@example.com", "role": "community_member", "is_active": true, "created_at":
"2026-05-10T12:17:05.849Z" }, { "user_id": 2, "username": "joy_tester", "email":
"joy@campuscare.edu", "role": "facility_manager", "is_active": true, "created_at": "2026-05-
10T11:51:37.193Z" } ] }
```

PUT <http://localhost:5000/api/admin/users/2/status>

Description: Activate or deactivate a user account

Authorization: Bearer ADMIN_TOKEN

Request Body Parameters: { "is_active": false }

Example Success Response (200 OK):

```
{ "message": "User account deactivated successfully", "user": { "user_id": 2, "username":
"joy_tester", "email": "joy@campuscare.edu", "role": "facility_manager", "is_active": false}}
```